实验目的：

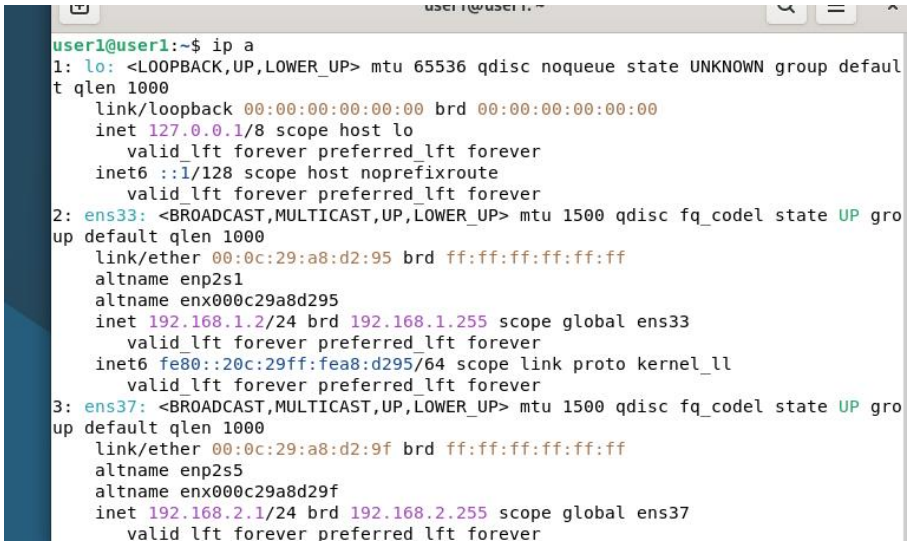
了解原始套接字的基本概念和使用方法。
掌握路由器进行 IP 数据报转发的基本原理。
实现于原始套接字的 IP 数据报的发送和接收
实现基于原始套接字的 IP 数据报转发，包括 AF_INET 和 AF_PACKET 原始套接字的应用。

实验内容：

(1)使用虚拟机实现多主机间的 UDP 数据报收发及转发
利用虚拟机搭建实验环境，掌握 Linux 下的 Socket 网络编程。
选做 1：改进程序，示例程序只实现了一个数据包（携带 1 条消息）的发、转、收过程，要求实现每条消息由控制台输入，并且不限制发送消息的数目。
(2)基于单网口主机的 IP 数据转发及收发
在局域网中，模拟 IP 数据报的路由转发过程。通过原始套接字实现了完整的数据封装过程，实现了 UDP 头部、IP 头部、MAC 帧头部的构造。
选做 2：扩展实验的网络规模，由原始方案中 3 台主机增加到不少于 5 台主机，共同完成 IP 数据报转发及收发过程，要求采用转发表改进示例程序，增加程序通用性。
(3) 基于双网口主机的路由转发
构造了静态路由表，并实现了不同子网间的 IP 数据报查表转发过程。
选做 3：通过完善路由表，改进示例程序实现双向传输。

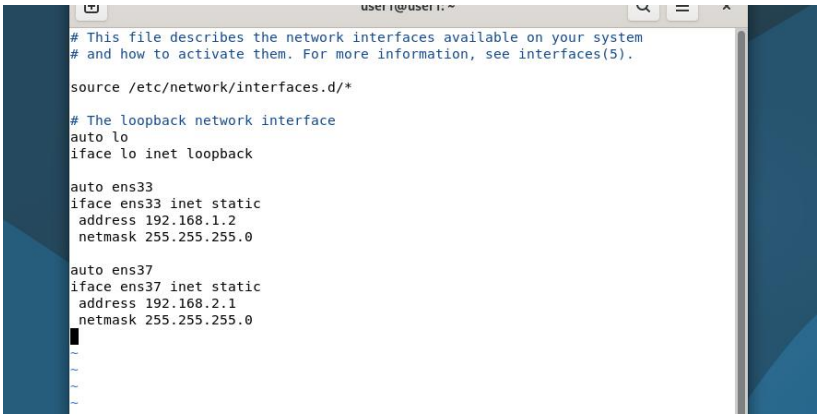
实验过程：

代码测试环境为Debian 13



首先先对实验环境进行配置，不管是哪个主机，都先使用ip a检测网络适配器名称及其状态，仅选择双网卡虚拟机作为演示图片，如图所示，图中显示了两个适配器状态，另外一个为本地回环连接，两个网络适配器的名称分别为ens33、ens37，为了方便实验，我们先使用su进入超级用户模式(需输入管理员密码)，然后使用sudo指令，以管理员权限运行vim，打开网络配置文件(指令为sudo vim /etc/network/interfaces，注意第一个斜杠不可省略，否则容易打

开并写入错误的文件)。



对于只有一个网卡的虚拟机，由于使用clone产生，因此设备名称均为ens33，一开始打开网络配置文件时只有本地环路链接，为了配置静态IP地址，需要按如图格式写入配置，address为选择的IP地址，netmask为子网掩码，其余只需要把ens33替换为设备名称即可。
为了防止ufw阻挡端口流量，我们还需要输入sudo ufw allow 12345/udp这样的指令来使得UDP流量可以通过防火墙。网络配置完毕以后，我们就可以开始编写与测试代码了。

验收内容1：

```c
void getFormattedTime(char* buffer, size_t buf_size) {
    time_t rawtime;
    struct tm *timeinfo;

    time(&rawtime);
    timeinfo = localtime(&rawtime);
    strftime(buffer, buf_size, "%Y-%m-%d %H:%M:%S", timeinfo);
}
```

获取当前时间字符串，在三个程序中是通用的。

Test1_send.c

```c
    int sockfd;
    struct sockaddr_in dest_addr;
    char buffer[1024];
    int port = 12345;

    const char *HOST_B_IP = "192.168.1.2";
    if ((sockfd = socket(AF_INET, SOCK_DGRAM, IPPROTO_UDP)) < 0) {
        perror("socket");
        return 1;
    }
    dest_addr.sin_family = AF_INET;
    dest_addr.sin_port = htons(port);
    dest_addr.sin_addr.s_addr = inet_addr(HOST_B_IP);
    printf("UDP Sender (Host A: 192.168.1.1) is running.\n");
    printf("Sending to Host B (%s:%d). Type 'exit' to quit:\n", HOST_B_IP,
```

```
port);
```

进行网络参数设置，循环输入要发送的内容直到输入 exit 为止。

Test1_forward.c

```
    while (1) {
        addr_len = sizeof(src_addr);
        int recv_len = recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0,
(struct sockaddr *)&src_addr, &addr_len);

        if (recv_len < 0) {
            perror("recvfrom");
            continue;
        }
        buffer[recv_len] = '\0';
        char time_buffer[80];
        memset(time_buffer, 0, sizeof(time_buffer));
        getFormattedTime(time_buffer, sizeof(time_buffer));
        printf("[%s] Received from %s: \"%s\". Forwarding to Host C...\n",
time_buffer, inet_ntoa(src_addr.sin_addr), buffer);

        if (sendto(sockfd, buffer, strlen(buffer), 0, (struct sockaddr
*)&dest_addr,
                   sizeof(dest_addr)) < 0) {
            perror("sendto");
        } else {
            printf("Datagram forwarded.\n");
        }
    }
```

无限循环，接收来自第一个主机发送的消息，打印并转发给目标主机。

Test1_recv.c

```
    while (1) {
        addr_len = sizeof(src_addr);
        int recv_len = recvfrom(sockfd, buffer, sizeof(buffer) - 1, 0,
(struct sockaddr *)&src_addr, &addr_len);

        if (recv_len < 0) {
            perror("recvfrom");
            continue;
        }
        buffer[recv_len] = '\0';
        char time_buffer[80];
        memset(time_buffer, 0, sizeof(time_buffer));
        getFormattedTime(time_buffer, sizeof(time_buffer));
```

```
        printf("[%s] Datagram received from %s: \"%s\"\n", time_buffer,
inet_ntoa(src_addr.sin_addr), buffer);
    }
```

接收和打印转发来的消息。

验收内容 2

utils.h

```
#ifndef UTILS_H
#define UTILS_H

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <sys/socket.h>
#include <sys/ioctl.h>
#include <net/if.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/udp.h>
#include <netinet/if_ether.h>
#include <net/ethernet.h>
#include <arpa/inet.h>
#include <linux/if_packet.h>
// --- 全局网络配置参数 (Global Network Configuration Parameters) ---
#define IFNAME "ens33"
// IP 数据报头中不变的源/目的 IP 地址
#define SRC_IP_STR "192.168.1.1"
#define DST_IP_STR "192.168.1.3"
// 默认的源/目的端口
#define SRC_PORT 54321
#define DST_PORT 12345
#define MAC_R_BYTES {0x00, 0x0C, 0x29, 0xA8, 0xD2, 0x95}
#define MAC_C_BYTES {0x00, 0x0C, 0x29, 0xBB, 0x20, 0xCA}
//IP 校验和
unsigned short csum(unsigned short *buf, int nwords) {
    unsigned long sum;
    for (sum = 0; nwords > 0; nwords--)
        sum += *buf++;
    sum = (sum >> 16) + (sum & 0xffff);
    sum += (sum >> 16);
    return (unsigned short)(~sum);
}
```

```
// 打印信息函数
void print_packet_info(unsigned char* buffer, int size, const char* role)
{
    struct ethhdr *eth = (struct ethhdr *)buffer;
    // 检查是否小于数据报最小大小
    if (size < sizeof(struct ethhdr) + sizeof(struct iphdr)) {
        printf("ERROR: Received packet size is too small.\n");
        return;
    }
    struct iphdr *iph = (struct iphdr *)(buffer + sizeof(struct ethhdr));

    // Check minimum size for UDP header
    if (size < sizeof(struct ethhdr) + iph->ihl * 4 + sizeof(struct udphdr))
{
        printf("ERROR: Received packet size is too small for UDP.\n");
        return;
    }
    struct udphdr *udph = (struct udphdr *)(buffer + sizeof(struct ethhdr)
+ iph->ihl * 4);

    // Timestamp
    time_t timer;
    char buffer_time[26];
    struct tm* tm_info;
    time(&timer);
    tm_info = localtime(&timer);
    strftime(buffer_time, 26, "%Y-%m-%d %H:%M:%S", tm_info);
    struct sockaddr_in source, dest;
    memset(&source, 0, sizeof(source));
    source.sin_addr.s_addr = iph->saddr;
    memset(&dest, 0, sizeof(dest));
    dest.sin_addr.s_addr = iph->daddr;

    char *data = (char *)(buffer + sizeof(struct ethhdr) + iph->ihl * 4 +
sizeof(struct udphdr));
    int data_len = ntohs(udph->len) - sizeof(struct udphdr);

    printf("\n--- %s: IP Datagram Received (Time: %s) ---\n", role,
buffer_time);

    printf("Ethernet Frame Header\n");
    printf("MAC | Src: %02X:%02X:%02X:%02X:%02X:%02X\n",
        eth->h_source[0], eth->h_source[1], eth->h_source[2],
```

```
eth->h_source[3], eth->h_source[4], eth->h_source[5]);
    printf("MAC | Dst: %02X:%02X:%02X:%02X:%02X:%02X\n",
        eth->h_dest[0], eth->h_dest[1], eth->h_dest[2], eth->h_dest[3],
eth->h_dest[4], eth->h_dest[5]);
    printf("EtherType: 0x%04X (IP)\n", ntohs(eth->h_proto));
    printf("IP  | Src: %s\n", inet_ntoa(source.sin_addr));
    printf("IP  | Dst: %s\n", inet_ntoa(dest.sin_addr));
    printf("Len : %d bytes\n", ntohs(iph->tot_len));
    printf("TTL : %d\n", iph->ttl);
    printf("Ptl : %d (UDP)\n", iph->protocol);
    printf("Csum: 0x%04X\n", ntohs(iph->check));
    printf("Port| Src: %d\n", ntohs(udph->source));
    printf("Port| Dst: %d\n", ntohs(udph->dest));

    printf("Payload (Data):\n");
    printf("Message:'%.*s'\n", data_len, data);
    printf("-------------------------------------\n");
}
```

打印数据报信息函数

```
#endif // UTILS_H
```

Test2_send.c

循环输入要转发的内容，设置好下一跳 MAC 地址，提前处理好数据报各段(以太网头部、IPv4 数据报头部、UDP 头部、数据起始位置，由外而内)指针位置

```
    int sd;
    while(1)
    {
        unsigned char buffer[1024];
        struct ethhdr *eth = (struct ethhdr *)buffer;
        struct iphdr *iph = (struct iphdr *)(buffer + sizeof(struct ethhdr));
        struct udphdr *udph = (struct udphdr *)(buffer + sizeof(struct
ethhdr) + sizeof(struct iphdr));
        char *data = (char *)(buffer + sizeof(struct ethhdr) + sizeof(struct
iphdr) + sizeof(struct udphdr));

        struct sockaddr_ll sll;
        struct ifreq ifr;

        unsigned char src_mac[6];
        unsigned char dest_mac[6] = MAC_R_BYTES; // 帧目标：路由器 R 的 MAC

        char message[1024];
        printf("> ");
        if (fgets(message, sizeof(message), stdin) == NULL) {
```

```
            break;
        }

        buffer[strcspn(message, "\n")] = 0;
```

去掉输入内容中的空格，如果出现 exit，退出发送。

```
        if (strcmp(message, "exit\n") == 0) {
            printf("Exiting sender...\n");
            break;
        }

        int payload_len = strlen(message);

        int total_len = sizeof(struct ethhdr) + sizeof(struct iphdr) +
sizeof(struct udphdr) + payload_len;

        // 1. 创建原始套接字 (Create Raw Socket)
        sd = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_IP));
        if (sd < 0) {
            perror("socket error");
            return 1;
        }

        // 获取接口索引和源 MAC 地址 (Get Interface Index and Source MAC)
        memset(&ifr, 0, sizeof(ifr));
        strncpy(ifr.ifr_name, IFNAME, IFNAMSIZ - 1); //IFNAME 为网卡名称
        if (ioctl(sd, SIOCGIFINDEX, &ifr) < 0) { //使用 ioctl 补充参数
            perror("ioctl SIOCGIFINDEX error");
            close(sd);
            return 1;
        }
        sll.sll_ifindex = ifr.ifr_ifindex; //sll 为以太网数据报

        if (ioctl(sd, SIOCGIFHWADDR, &ifr) < 0) { //获取 MAC 地址
            perror("ioctl SIOCGIFHWADDR error");
            close(sd);
            return 1;
        }
        memcpy(src_mac, ifr.ifr_hwaddr.sa_data, 6);

        // 2. 构造数据报 (Construct Packet)
        memset(buffer, 0, 1024);

        //  2.1. UDP Header (传输层)
        udph->source = htons(SRC_PORT);
```

```
        udph->dest = htons(DST_PORT); // * 使用 12345 端口 *
        udph->len = htons(sizeof(struct udphdr) + payload_len);
        udph->check = 0;
        strcpy(data, message); // 载荷

        //  2.2. IP Header (网络层)
        iph->ihl = 5;
        iph->version = 4;
        iph->tos = 0;
        iph->tot_len = htons(sizeof(struct iphdr) + sizeof(struct udphdr)
+ payload_len);
        iph->id = htons(rand());
        iph->frag_off = 0;
        iph->ttl = 64;      // 初始 TTL
        iph->protocol = IPPROTO_UDP;
        iph->saddr = inet_addr(SRC_IP_STR); // 源 IP 不变
        iph->daddr = inet_addr(DST_IP_STR); // 目的 IP 不变
        iph->check = 0;
        iph->check = csum((unsigned short *)iph, sizeof(struct iphdr) / 2);
// 计算 IP 校验和

        //  2.3. MAC Frame Header (数据链路层)
        memcpy(eth->h_source, src_mac, 6);     // 源 MAC: Host A's MAC
        memcpy(eth->h_dest, dest_mac, 6);      // 目的 MAC: Router R's MAC
(下一跳)
        eth->h_proto = htons(ETH_P_IP);        // 协议类型: IP

        // 3. 准备发送地址信息 (Prepare Socket Address)
        sll.sll_family = AF_PACKET;
        sll.sll_halen = ETH_ALEN;
        sll.sll_protocol = htons(ETH_P_IP);
        memcpy(sll.sll_addr, dest_mac, 6);

        // 4. 发送数据 (Send Packet)
        if (sendto(sd, buffer, total_len, 0, (struct sockaddr*)&sll,
sizeof(sll)) < 0) {
            perror("sendto error");
            return 1;
        }

        printf("Host A (Sender) Status:\n");
        printf("  Datagram sent successfully.\n");
        printf("  IP Header: %s -> %s\n", SRC_IP_STR, DST_IP_STR);
        printf("  UDP Port: %d -> %d\n", SRC_PORT, DST_PORT);
```

```
    }

    close(sd);
    return 0;
```

Test2_router.c

依旧循环接收，但是采用的涉及到数据报首部信息的改变，包括源 MAC 地址与目的 MAC 地址、TTL(数据报生存时间，且注意若 TTL <= 1 说明生存时间到头，丢弃该包)和检验和(因为 TTL 被修改)，随后转发数据报。由于网络中充斥大量非指定程序发送的数据报，因此添加了过滤机制，便于观察，即比较源 IP 地址、目的 IP 地址、MAC 地址。

```
    while (1) {
        socklen_t sll_len = sizeof(sll_recv);
        int len = recvfrom(sd_recv, buffer, 65536, 0, (struct
sockaddr*)&sll_recv, &sll_len);
        if (len <= 0) {
            continue;
        }

        struct ethhdr *eth = (struct ethhdr *)buffer;

        //  过滤 (Filtering)
        // 1. 过滤非 IPv4 帧 或 目的 MAC 不匹配
        if (ntohs(eth->h_proto) != ETH_P_IP || memcmp(eth->h_dest,
router_mac, 6) != 0) {
            continue;
        }

        struct iphdr *iph = (struct iphdr *)(buffer + sizeof(struct ethhdr));

        // 2. 过滤 IP 地址不匹配 或 非 UDP 协议
        if (iph->daddr != target_dst_ip_addr || iph->saddr !=
target_src_ip_addr || iph->protocol != IPPROTO_UDP) {
            continue;
        }

        // 检查 UDP 头部长度
        if (len < sizeof(struct ethhdr) + iph->ihl * 4 + sizeof(struct
udphdr)) {
            continue;
        }
        struct udphdr *udph = (struct udphdr *)(buffer + sizeof(struct
ethhdr) + iph->ihl * 4);

        // 3. 过滤目的端口不匹配
```

```
        if (udph->dest != target_dst_port) {
            continue;
        }

        // 1. 展示收到数据报信息 (Display Received Packet Info)
        print_packet_info(buffer, len, "Host R (Router) - Datagram
Received");

        //  IP 数据报转发逻辑 (IP Data Forwarding Logic)

        //  (新增) 展示修改前的数据报关键信息
        printf("\n Host R: Processing Changes BEFORE Forwarding \n");
        printf("  Current TTL: %d\n", iph->ttl);
        printf("  Current Src MAC: %02X:%02X:%02X:%02X:%02X:%02X\n",
            eth->h_source[0], eth->h_source[1], eth->h_source[2],
eth->h_source[3], eth->h_source[4], eth->h_source[5]);
        printf("  Current Dst MAC: %02X:%02X:%02X:%02X:%02X:%02X\n",
            eth->h_dest[0], eth->h_dest[1], eth->h_dest[2],
eth->h_dest[3], eth->h_dest[4], eth->h_dest[5]);
        printf("-\n");

        // 2. 执行 IP 头部修改
        if (iph->ttl <= 1) {
            printf("WARNING: TTL <= 1, Datagram Dropped.\n");
            continue;
        }

        unsigned short old_checksum = iph->check;

        iph->ttl--;       // TTL 减 1 (Decrement TTL)
        iph->check = 0;   // 清零校验和 (Reset Checksum)
        iph->check = csum((unsigned short *)iph, iph->ihl * 2); // 重新计
算 IP 校验和 (Recalculate Checksum)

        // 3. 重新构造 MAC 帧头部
        memcpy(eth->h_source, router_mac, 6);   // 新的源 MAC: 路由器 R 的
MAC
        memcpy(eth->h_dest, receiver_mac, 6);   // 新的目的 MAC: 接收者 C
的 MAC
```

```
        //  (新增) 展示修改后的数据报关键信息
        printf("\n Host R: Modifications Completed \n");
        printf("  TTL Changed: %d -> %d\n", iph->ttl + 1, iph->ttl);
        printf("  Checksum Changed: 0x%04X -> 0x%04X\n",
```

```
ntohs(old_checksum), ntohs(iph->check));
        printf("  New Src MAC: %02X:%02X:%02X:%02X:%02X:%02X\n",
            eth->h_source[0], eth->h_source[1], eth->h_source[2],
eth->h_source[3], eth->h_source[4], eth->h_source[5]);
        printf("  New Dst MAC: %02X:%02X:%02X:%02X:%02X:%02X\n",
            eth->h_dest[0], eth->h_dest[1], eth->h_dest[2],
eth->h_dest[3], eth->h_dest[4], eth->h_dest[5]);
        printf("\n");

        // 4. 发送转发后的数据报
        sll_send.sll_family = AF_PACKET;
        sll_send.sll_halen = ETH_ALEN;
        sll_send.sll_protocol = htons(ETH_P_IP);
        memcpy(sll_send.sll_addr, receiver_mac, 6);
        if (sendto(sd_send, buffer, len, 0, (struct sockaddr*)&sll_send,
sizeof(sll_send)) < 0) {
            perror("sendto error");
            continue;
        }
        printf("Host R: Forwarding successful!\n");
    }
```

Test2_recv.c

```
    while (1) {
        socklen_t sll_len = sizeof(sll);
        int len = recvfrom(sd, buffer, 1024, 0, (struct sockaddr*)&sll,
&sll_len);
        if (len <= 0) {
            continue;
        }

        struct ethhdr *eth = (struct ethhdr *)buffer;

        //  过滤 (Filtering)
        // 1. 过滤非 IPv4 帧
        if (ntohs(eth->h_proto) != ETH_P_IP) {
            continue;
        }
        // 2. 过滤非发给接收者 MAC 的帧
        if (memcmp(eth->h_dest, receiver_mac, 6) != 0) {
            continue;
        }

        struct iphdr *iph = (struct iphdr *)(buffer + sizeof(struct ethhdr));
```

```
        // 3. 过滤非目标 IP (源 IP 和 目的 IP 必须匹配)
        if (iph->daddr != target_dst_ip_addr || iph->saddr !=
target_src_ip_addr) {
            continue;
        }
        // 4. 过滤非 UDP 协议
        if (iph->protocol != IPPROTO_UDP) {
            continue;
        }
        // 获取 UDP 头部 (Get UDP header)
        if (len < sizeof(struct ethhdr) + iph->ihl * 4 + sizeof(struct
udphdr)) {
            continue;
        }
        struct udphdr *udph = (struct udphdr *)(buffer + sizeof(struct
ethhdr) + iph->ihl * 4);

        // 5. 过滤非目标端口
        if (udph->dest != target_dst_port) {
            continue;
        }
        // 2. 展示收到数据报信息 (Display Received Packet Info)
        print_packet_info(buffer, len, "Host C (Receiver) - Matched and
Accepted");

        printf("Host C: Datagram successfully received and processed.\n");
```

过滤数据报并打印消息。

验收内容 3

公共头文件 utils1.h

其实与 utils.h 大同小异，主要修改了目的 IP 地址和增加相关主机的 MAC 地址。

```
typedef struct {
    char dest_net[16];          // 目的网络 (e.g., "192.168.2.0")
    char netmask[16];           // 子网掩码 (e.g., "255.255.255.0")
    char next_hop_ip[16];       // 下一跳 IP (如果是直连，则为 "0.0.0.0")
    char interface[IFNAMSIZ];   // 出口接口 (e.g., IFNAME_2)
} RoutingEntry;

// ARP 表项 (模拟): 根据 IP 查找 MAC 地址
typedef struct {
    char ip_addr[16];
    unsigned char mac_addr[6];
} ArpEntry;
```

Test3_send.c、Test3_recv.c 基本与实验 2 相差无几。
主要变化发生在中间的路由主机上

```
// router_r1.c
#include "utils1.h"

// R1 配置
#define ROUTER_MAC MAC_R1_BYTES
#define ROUTER_NAME "Host R1 (Router 1)"
#define NEXT_HOP_MAC MAC_R2_BYTES // 下一跳是 R2
```

```
// R1 的转发表：将 FINAL_DST_IP_STR 路由到 R2
RoutingEntry routing_table[] = {
    {FINAL_DST_IP_STR, NEXT_HOP_MAC, IFNAME}
};
const int ROUTING_TABLE_SIZE = sizeof(routing_table) /
sizeof(RoutingEntry);
```

新增静态路由表

```
// 路由表查找函数 (Route lookup function)
RoutingEntry* route_lookup(unsigned int dest_ip) {
    char dest_ip_str[16];
    struct in_addr addr;
    addr.s_addr = dest_ip;
    strncpy(dest_ip_str, inet_ntoa(addr), 16);
    for (int i = 0; i < ROUTING_TABLE_SIZE; i++) {
        if (strcmp(routing_table[i].dest_ip, dest_ip_str) == 0) {
            return &routing_table[i];
        }
    }
    return NULL;
}
```

路由查询，目的 IP 地址与路由表中保存的 IP 地址相符合，就保存并发向对应的 MAC 地址。

```
// 主程序 (Router R1)
int main() {
    int sd_recv, sd_send;
    unsigned char buffer[65536];
    struct sockaddr_ll sll_recv, sll_send;
    struct ifreq ifr;

    unsigned char router_mac[6] = ROUTER_MAC;
    unsigned int target_src_ip_addr = inet_addr(FINAL_SRC_IP_STR);
    unsigned short target_dst_port = htons(DST_PORT);
```

```
    // 1. 创建套接字 (Socket setup, omitted for brevity but required)
```

```
    sd_recv = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
    sd_send = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_IP));

    memset(&ifr, 0, sizeof(ifr));
    strncpy(ifr.ifr_name, IFNAME, IFNAMSIZ - 1);
    ioctl(sd_send, SIOCGIFINDEX, &ifr);
    sll_send.sll_ifindex = ifr.ifr_ifindex;

    // 打印路由表信息
    printf("%s Loaded Routing Table (%d entries):\n", ROUTER_NAME,
ROUTING_TABLE_SIZE);
    for (int i = 0; i < ROUTING_TABLE_SIZE; i++) {
        printf("  Entry %d: Dst IP: %s -> Next Hop
MAC: %02X:%02X:%02X:%02X:%02X:%02X\n",
           i, routing_table[i].dest_ip,
routing_table[i].next_hop_mac[0], routing_table[i].next_hop_mac[1],
routing_table[i].next_hop_mac[2], routing_table[i].next_hop_mac[3],
routing_table[i].next_hop_mac[4], routing_table[i].next_hop_mac[5]);
    }
    printf("%s is listening on %s...\n", ROUTER_NAME, IFNAME);
```

```
    // 主循环: 接收 -> 检查 -> 查表 -> 转发
    while (1) {
        socklen_t sll_len = sizeof(sll_recv);
        int len = recvfrom(sd_recv, buffer, 65536, 0, (struct
sockaddr*)&sll_recv, &sll_len);
        if (len <= 0) {
            continue;
        }

        struct ethhdr *eth = (struct ethhdr *)buffer;

        // --- 过滤 (Filtering) ---
        // L2/L3/L4 过滤逻辑 (MAC, IP, Port)
        if (ntohs(eth->h_proto) != ETH_P_IP || memcmp(eth->h_dest,
router_mac, 6) != 0) {
            continue;
        }

        struct iphdr *iph = (struct iphdr *)(buffer + sizeof(struct ethhdr));
```

```
        if (iph->saddr != target_src_ip_addr || iph->protocol !=
IPPROTO_UDP) {
            continue;
```

```
        }

        if (len < sizeof(struct ethhdr) + iph->ihl * 4 + sizeof(struct
udphdr)) {
            continue;
        }
        struct udphdr *udph = (struct udphdr *)(buffer + sizeof(struct
ethhdr) + iph->ihl * 4);
        if (udph->dest != target_dst_port) {
            continue;
        }

        //查表转发
        RoutingEntry* route = route_lookup(iph->daddr);
        if (route == NULL) {
            printf("%s: ERROR: Destination IP %s not found in routing table.
Datagram Dropped.\n",
                   ROUTER_NAME, inet_ntoa(*(struct in_addr
*)&iph->daddr));
            continue;
        }

        // 1. 展示收到数据报信息 print_packet_info(buffer, len,
ROUTER_NAME);

        // 展示修改前后的数据报关键信息
        printf("\n--- %s: Processing Changes BEFORE Forwarding (Next Hop:
R2) ---\n", ROUTER_NAME);
        printf("  Current TTL: %d | Checksum: 0x%04X\n", iph->ttl,
ntohs(iph->check));
        printf("  MAC Dst: %02X:%02X:%02X:%02X:%02X:%02X (Self)\n",
eth->h_dest[0], eth->h_dest[1], eth->h_dest[2], eth->h_dest[3],
eth->h_dest[4], eth->h_dest[5]);
        printf("-----------------------------------------------------\n")
;

        // 2. 执行 IP 头部修改
        if (iph->ttl <= 1) {
            printf("WARNING: TTL <= 1, Datagram Dropped.\n");
            continue;
        }

        unsigned short old_checksum = iph->check;
        iph->ttl--;
```

```
        iph->check = 0;
        iph->check = csum((unsigned short *)iph, iph->ihl * 2);

        // 3. 重新构造 MAC 帧头部 (使用查表结果)
        memcpy(eth->h_source, router_mac, 6);               // 新的源 MAC: R1 的 MAC
        memcpy(eth->h_dest, route->next_hop_mac, 6);        // 新的目的 MAC: R2 的 MAC

        printf("\n--- %s: Modifications Completed (Ready to send to R2) ---\n", ROUTER_NAME);
        printf("  TTL Changed: %d -> %d\n", iph->ttl + 1, iph->ttl);
        printf("  Checksum Changed: 0x%04X -> 0x%04X\n", ntohs(old_checksum), ntohs(iph->check));
        printf("  MAC Dst: %02X:%02X:%02X:%02X:%02X:%02X (Next Hop R2)\n", eth->h_dest[0], eth->h_dest[1], eth->h_dest[2], eth->h_dest[3], eth->h_dest[4], eth->h_dest[5]);
        printf("---------------------------------------\n");

        // 4. **发送转发后的数据报**
        sll_send.sll_family = AF_PACKET;
        sll_send.sll_halen = ETH_ALEN;
        sll_send.sll_protocol = htons(ETH_P_IP);
        memcpy(sll_send.sll_addr, route->next_hop_mac, 6);
```

```
        if (sendto(sd_send, buffer, len, 0, (struct sockaddr*)&sll_send, sizeof(sll_send)) < 0) {
            perror("sendto error");
            continue;
        }
        printf("%s: Forwarding successful to Dst IP %s!\n", ROUTER_NAME, route->dest_ip);
    }
```

```
    close(sd_recv);
    close(sd_send);
    return 0;
}
```

其余路由程序代码基本相同，只需要修改路由表就可以了。

验收内容4

Test4_send.c

```
// host_a_udp.c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>

#define HOST_A_IP "192.168.1.1"
#define PORT_A 54321
#define HOST_C_IP "192.168.2.2"
#define PORT_C 12345
```

```
int main() {
    int sd;
    struct sockaddr_in serv_addr, cli_addr;
    char buffer[1024];

    // 1. 创建 UDP 套接字
    sd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sd < 0) {
        perror("socket error");
        return 1;
    }
```

```
    // 2. 绑定到 Host A 的 IP 和端口
    memset(&cli_addr, 0, sizeof(cli_addr));
    cli_addr.sin_family = AF_INET;
    cli_addr.sin_port = htons(PORT_A);
    cli_addr.sin_addr.s_addr = inet_addr(HOST_A_IP);
```

```
    if (bind(sd, (struct sockaddr*)&cli_addr, sizeof(cli_addr)) < 0) {
        perror("bind error");
        close(sd);
        return 1;
    }

    printf("Host A (%s:%d) is online.\n", HOST_A_IP, PORT_A);
```

```
    // 3. 准备目标地址 (Host C)
```

```
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT_C);
    serv_addr.sin_addr.s_addr = inet_addr(HOST_C_IP);
    // 4. 发送消息
    while(1)
    {
        printf("> ");
        if (fgets(buffer, sizeof(buffer), stdin) == NULL) {
            break;
        }
        buffer[strcspn(buffer, "\n")] = 0;
        if (strcmp(buffer, "exit") == 0) {
            printf("Exiting sender...\n");
        }
        if (sendto(sd, buffer, strlen(buffer), 0, (struct
sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
            perror("sendto error");
            close(sd);
            return 1;
        }
        printf("Host A: Sent message to %s: '%s'\n", HOST_C_IP, buffer);

        // 5. 等待回复
        printf("Host A: Waiting for reply...\n");
        socklen_t len = sizeof(serv_addr);
        int n = recvfrom(sd, buffer, 1024, 0, (struct sockaddr*)&serv_addr,
&len);
        if (n < 0) {
            perror("recvfrom error");
            close(sd);
            return 1;
        }

        buffer[n] = '\0';
        printf("Host A: Received reply from %s: '%s'\n",
inet_ntoa(serv_addr.sin_addr), buffer);

    }

    close(sd);
    return 0;
}
```

一个很基础的UDP接收和发送程序

Test4_recv.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <arpa/inet.h>
#include <unistd.h>

#define HOST_C_IP "192.168.2.2"
#define PORT_C 12345

int main() {
    int sd;
    struct sockaddr_in serv_addr, cli_addr;
    char buffer[1024];

    // 1. 创建 UDP 套接字
    sd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sd < 0) {
        perror("socket error");
        return 1;
    }

    // 2. 绑定到 Host C 的 IP 和端口
    memset(&serv_addr, 0, sizeof(serv_addr));
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT_C);
    serv_addr.sin_addr.s_addr = inet_addr(HOST_C_IP);
    if (bind(sd, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) < 0) {
        perror("bind error");
        close(sd);
        return 1;
    }

    printf("Host C (%s:%d) is online. Waiting for message...\n", HOST_C_IP,
PORT_C);
    // 3. 循环等待消息
    while (1) {
        socklen_t len = sizeof(cli_addr);
        int n = recvfrom(sd, buffer, 1024, 0, (struct sockaddr*)&cli_addr,
&len);
        if (n < 0) {
            perror("recvfrom error");
```

```
            continue;
        }
        buffer[n] = '\0';
        printf("Host C: Received message from %s: '%s'\n",
inet_ntoa(cli_addr.sin_addr), buffer);
        // 4. 发送回复
        const char* msg_to_a = "Hello! Got your message loud and clear.";
        if (sendto(sd, msg_to_a, strlen(msg_to_a), 0, (struct
sockaddr*)&cli_addr, sizeof(cli_addr)) < 0) {
            perror("sendto reply error");
        } else {
            printf("Host C: Sent reply to %s.\n",
inet_ntoa(cli_addr.sin_addr));
        }
    }

    close(sd);
    return 0;
}
```

与发送函数差别不大，主要是修改了参数与固定回复。

utils2.h

```
// utils.h

#ifndef UTILS_H
#define UTILS_H
```

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <time.h>
#include <sys/socket.h>
#include <sys/ioctl.h>
#include <sys/select.h> // 用于 select()
#include <net/if.h>
#include <netinet/in.h>
#include <netinet/ip.h>
#include <netinet/udp.h>
#include <netinet/if_ether.h>
#include <net/ethernet.h>
#include <arpa/inet.h>
#include <linux/if_packet.h>
```

```
// --- 全局网络配置参数 (Global Network Configuration Parameters) ---

#define IFNAME_1 "ens33"
// 路由器 R 的 eth1
#define IFNAME_2 "ens37"

// 端口配置 (用于过滤)
#define PORT_A 54321 // Host A 监听的端口
#define PORT_C 12345 // Host C 监听的端口

// IP 地址配置
#define HOST_A_IP "192.168.1.1"
#define ROUTER_R1_IP "192.168.1.2" // Router (eth0) IP
#define ROUTER_R2_IP "192.168.2.1" // Router (eth1) IP
#define HOST_C_IP "192.168.2.2"

// MAC 地址配置 (*** 必须替换为您的实际 MAC 地址 ***)
#define MAC_A_BYTES {0x00, 0x0C, 0x29, 0x14, 0x13, 0x10}     // Host A
#define MAC_R1_BYTES {0x00, 0x0C, 0x29, 0xA8, 0xD2, 0x95}   // Router (eth0)
#define MAC_R2_BYTES {0x00, 0x0C, 0x29, 0xA8, 0xD2, 0x9F}   // Router (eth1)
#define MAC_C_BYTES {0x00, 0x0c, 0x29, 0xBB, 0x20, 0xD4}     // Host C

// --- 路由表和 ARP 表结构 ---

// 路由表项：根据目的网络 IP 查找下一跳 IP 和出口
typedef struct {
    char dest_net[16];          // 目的网络 (e.g., "192.168.2.0")
    char netmask[16];           // 子网掩码 (e.g., "255.255.255.0")
    char next_hop_ip[16];       // 下一跳 IP (如果是直连，则为 "0.0.0.0")
    char interface[IFNAMSIZ];   // 出口接口 (e.g., IFNAME_2)
} RoutingEntry;

// ARP 表项 (模拟)：根据 IP 查找 MAC 地址
typedef struct {
    char ip_addr[16];
    unsigned char mac_addr[6];
} ArpEntry;

// IP 头部校验和计算函数 (Checksum function)
unsigned short csum(unsigned short *buf, int nwords) {
    unsigned long sum;
    for (sum = 0; nwords > 0; nwords--)
        sum += *buf++;
    sum = (sum >> 16) + (sum & 0xffff);
```

```
    sum += (sum >> 16);
    return (unsigned short)(~sum);
}
```

```
// 打印数据报详细信息
void print_packet_info(unsigned char* buffer, int size, const char* role)
{
    struct ethhdr *eth = (struct ethhdr *)buffer;
    if (ntohs(eth->h_proto) != ETH_P_IP) return;
    struct iphdr *iph = (struct iphdr *)(buffer + sizeof(struct ethhdr));
    if (iph->protocol != IPPROTO_UDP) return;
    struct udphdr *udph = (struct udphdr *)(buffer + sizeof(struct ethhdr)
+ iph->ihl * 4);

    // 时间戳
    time_t timer;
    char buffer_time[26];
    struct tm* tm_info;
    time(&timer);
    tm_info = localtime(&timer);
    strftime(buffer_time, 26, "%Y-%m-%d %H:%M:%S", tm_info);
```

```
    struct sockaddr_in source, dest;
    memset(&source, 0, sizeof(source));
    source.sin_addr.s_addr = iph->saddr;
    memset(&dest, 0, sizeof(dest));
    dest.sin_addr.s_addr = iph->daddr;

    printf("\n--- %s: IP Datagram Captured (Time: %s) ---\n", role,
buffer_time);

    printf("  L2 - Source MAC:      %02X:%02X:%02X:%02X:%02X:%02X\n",
        eth->h_source[0], eth->h_source[1], eth->h_source[2],
eth->h_source[3], eth->h_source[4], eth->h_source[5]);
    printf("  L2 - Destination MAC: %02X:%02X:%02X:%02X:%02X:%02X\n",
        eth->h_dest[0], eth->h_dest[1], eth->h_dest[2], eth->h_dest[3],
eth->h_dest[4], eth->h_dest[5]);
    printf("  L3 - Source IP:       %s\n", inet_ntoa(source.sin_addr));
    printf("  L3 - Destination IP:  %s\n", inet_ntoa(dest.sin_addr));
    printf("  L3 - TTL:             %d\n", iph->ttl);
    printf("  L4 - Source Port:     %d\n", ntohs(udph->source));
    printf("  L4 - Destination Port: %d\n", ntohs(udph->dest));
    printf("-------------------------------------------------\n");
}
```

```
// 辅助函数：创建并绑定到特定接口的 AF_PACKET 套接字
int create_raw_socket(const char* interface_name, int* if_index_out,
unsigned char* mac_out) {
    int sd;
    struct ifreq ifr;
    struct sockaddr_ll sll;
    sd = socket(AF_PACKET, SOCK_RAW, htons(ETH_P_ALL));
    if (sd < 0) {
        perror("socket error");
        return -1;
    }
    // 获取接口索引
    memset(&ifr, 0, sizeof(ifr));
    strncpy(ifr.ifr_name, interface_name, IFNAMSIZ - 1);
    if (ioctl(sd, SIOCGIFINDEX, &ifr) < 0) {
        perror("ioctl SIOCGIFINDEX error");
        close(sd);
        return -1;
    }
    *if_index_out = ifr.ifr_ifindex;
    // 获取接口 MAC 地址
    if (ioctl(sd, SIOCGIFHWADDR, &ifr) < 0) {
        perror("ioctl SIOCGIFHWADDR error");
        close(sd);
        return -1;
    }
    memcpy(mac_out, ifr.ifr_hwaddr.sa_data, 6);
    // 绑定套接字到特定接口
    memset(&sll, 0, sizeof(sll));
    sll.sll_family = AF_PACKET;
    sll.sll_ifindex = *if_index_out;
    sll.sll_protocol = htons(ETH_P_ALL);
    if (bind(sd, (struct sockaddr*)&sll, sizeof(sll)) < 0) {
        perror("bind error");
        close(sd);
        return -1;
    }

    return sd;
}

#endif // UTILS_H
```

Test4_router.c

```
// router_r.c
#include "utils2.h"

//  路由表和 ARP 表配置

// 静态路由表
RoutingEntry routing_table[] = {
    // 目的网络         子网掩码            下一跳          出口
    {"192.168.1.0", "255.255.255.0", "0.0.0.0", IFNAME_1}, // 直连
    {"192.168.2.0", "255.255.255.0", "0.0.0.0", IFNAME_2}  // 直连
};
const int ROUTING_TABLE_SIZE = 2;

// 静态 ARP 表（模拟）
ArpEntry arp_table[] = {
    {HOST_A_IP, MAC_A_BYTES},
    {HOST_C_IP, MAC_C_BYTES}
};
const int ARP_TABLE_SIZE = 2;

//  辅助函数

// 路由表查找（返回出口接口名称）
const char* route_lookup(unsigned int dest_ip_addr) {
    for (int i = 0; i < ROUTING_TABLE_SIZE; i++) {
        unsigned int net_addr = inet_addr(routing_table[i].dest_net);
        unsigned int mask_addr = inet_addr(routing_table[i].netmask);

        if ((dest_ip_addr & mask_addr) == net_addr) {
            return routing_table[i].interface;
        }
    }
    return NULL; // 没有路由
}

// ARP 表查找（返回下一跳 MAC）
unsigned char* arp_lookup(unsigned int next_hop_ip_addr) {
    struct in_addr addr;
    addr.s_addr = next_hop_ip_addr;
    char ip_str[16];
    strcpy(ip_str, inet_ntoa(addr));

    for (int i = 0; i < ARP_TABLE_SIZE; i++) {
```

```
        if (strcmp(arp_table[i].ip_addr, ip_str) == 0) {
            return arp_table[i].mac_addr;
        }
    }
    return NULL; // 没有 MAC
}

//  路由器转发核心逻辑
void handle_packet(int sd_in, int sd_out, int if_index_out, unsigned char*
mac_out, const char* role_name) {

    unsigned char buffer[65536];
    int len = recvfrom(sd_in, buffer, 65536, 0, NULL, NULL);
    if (len <= 0) return;

    struct enshdr *ens = (struct enshdr *)buffer;

    // 1. 过滤 L2 (确保是 IP 包)
    if (ntohs(ens->h_proto) != Ens_P_IP) return;

    struct iphdr *iph = (struct iphdr *)(buffer + sizeof(struct enshdr));

    // 2. 路由查找：根据目的 IP 查找路由表
    const char* out_if_name = route_lookup(iph->daddr);

    // 3. ARP 查找：查找下一跳 MAC
    // (在直连路由中，下一跳 IP 就是最终目的 IP)
    unsigned char* next_hop_mac = arp_lookup(iph->daddr);

    // 4. 检查是否需要转发 (如果找不到路由或 ARP，则丢弃)
    if (out_if_name == NULL || next_hop_mac == NULL) {
        // printf("%s: No route or ARP entry found. Dropping packet.\n",
role_name);
        return;
    }

    // 5. 打印详细日志 (满足验收要求)
    print_packet_info(buffer, len, role_name);

    // 6. 执行转发
    // 检查 TTL
    if (iph->ttl <= 1) {
        printf("%s: WARNING: TTL <= 1, Datagram Dropped.\n", role_name);
        return;
```

```
    }

    // 修改 L3 (IP Header)
    iph->ttl--;
    iph->check = 0;
    iph->check = csum((unsigned short *)iph, iph->ihl * 2);

    // 修改 L2 (Ensernet Header)
    memcpy(ens->h_source, mac_out, 6);   // 源 MAC 替换为出口 MAC
    memcpy(ens->h_dest, next_hop_mac, 6); // 目的 MAC 替换为下一跳 MAC

    // 7. 准备发送
    struct sockaddr_ll sll_send;
    memset(&sll_send, 0, sizeof(sll_send));
    sll_send.sll_family = AF_PACKET;
    sll_send.sll_ifindex = if_index_out;
    sll_send.sll_halen = Ens_ALEN;
    sll_send.sll_protocol = htons(Ens_P_IP);
    memcpy(sll_send.sll_addr, next_hop_mac, 6);

    if (sendto(sd_out, buffer, len, 0, (struct sockaddr*)&sll_send,
sizeof(sll_send)) < 0) {
        perror("sendto error");
    } else {
        print_packet_info(buffer, len, role_name);
    }
}

//主程序
int main() {
    int sd1, sd2; // 套接字 1 (ens0) 和 套接字 2 (ens1)
    int if_index1, if_index2;
    unsigned char mac1[6];
    unsigned char mac2[6];

    // 使用辅助函数创建并绑定套接字
    sd1 = create_raw_socket(IFNAME_1, &if_index1, mac1);
    if (sd1 < 0) {
        fprintf(stderr, "Error creating socket for %s\n", IFNAME_1);
        return 1;
    }

    sd2 = create_raw_socket(IFNAME_2, &if_index2, mac2);
    if (sd2 < 0) {
```

```
        fprintf(stderr, "Error creating socket for %s\n", IFNAME_2);
        return 1;
    }

    printf("Router R (Dual-NIC) is online... (Kernel forwarding
disabled)\n");
    printf("  Listening on %s (MAC: %02X:%02X:%02X:%02X:%02X:%02X)\n",
IFNAME_1, mac1[0], mac1[1], mac1[2], mac1[3], mac1[4], mac1[5]);
    printf("  Listening on %s (MAC: %02X:%02X:%02X:%02X:%02X:%02X)\n",
IFNAME_2, mac2[0], mac2[1], mac2[2], mac1[3], mac1[4], mac1[5]);
```

```
    // 使用 select() 监听两个套接字
    fd_set read_fds;
    int max_fd = (sd1 > sd2) ? sd1 : sd2;
```

```
    while (1) {
        FD_ZERO(&read_fds);
        FD_SET(sd1, &read_fds);
        FD_SET(sd2, &read_fds);
```

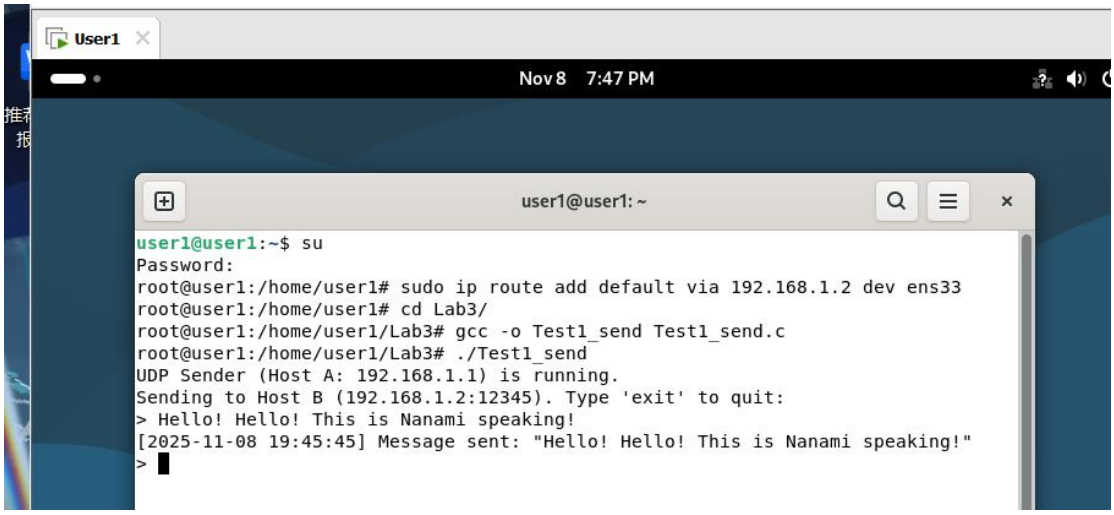
```
        if (select(max_fd + 1, &read_fds, NULL, NULL, NULL) < 0) {
            perror("select error");
            break;
        }
```

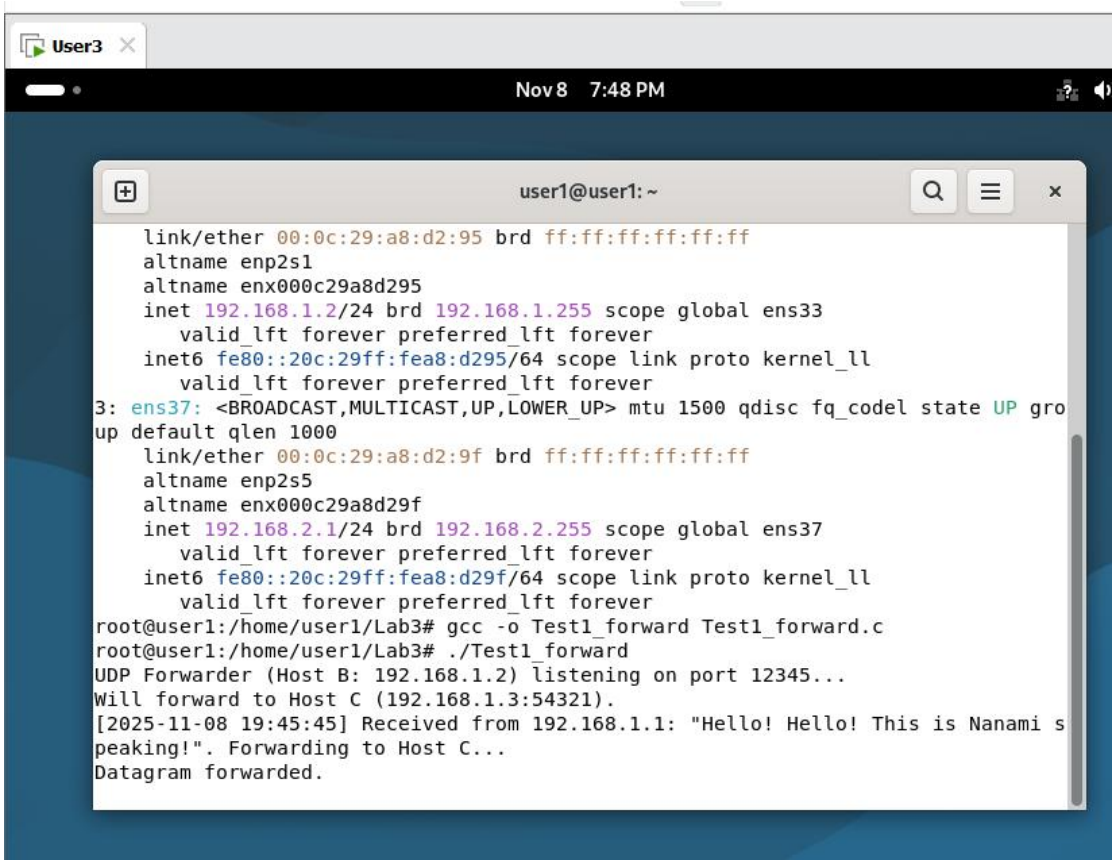
```
        // 检查是否从接口 1 (ens0) 收到数据
        if (FD_ISSET(sd1, &read_fds)) {
            // 收到来自 192.168.1.0 子网的数据，转发到 接口 2 (ens1)
            // (mac2 是 ens1 的 MAC，将作为转发包的源 MAC)
            handle_packet(sd1, sd2, if_index2, mac2, "Router (IF1->IF2)");
        }
```

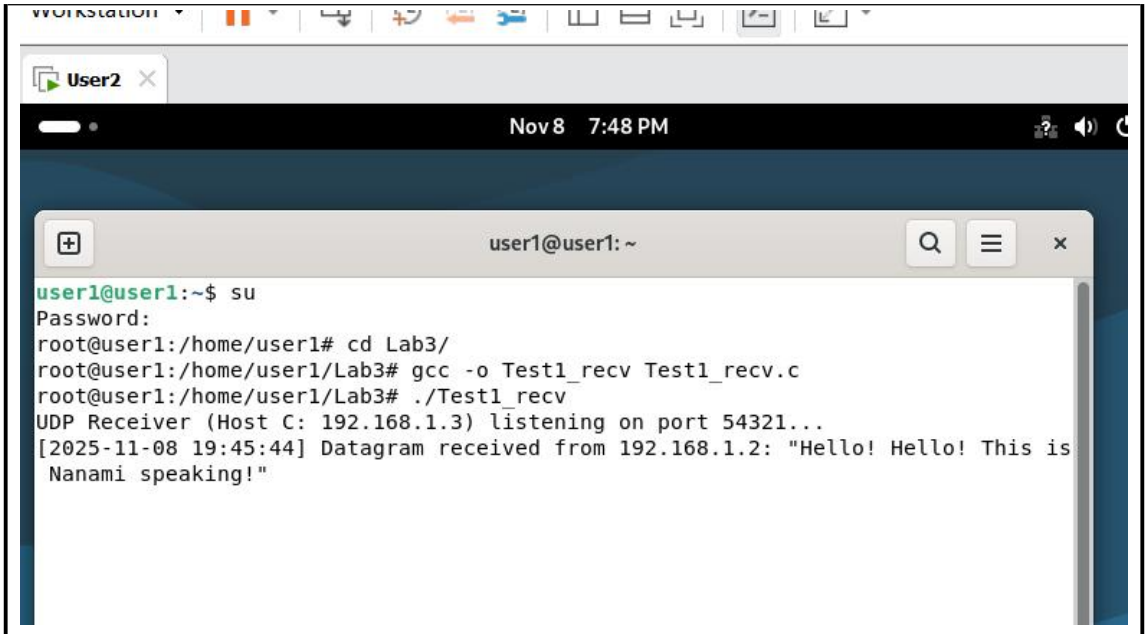
```
        // 检查是否从接口 2 (ens1) 收到数据
        if (FD_ISSET(sd2, &read_fds)) {
            // 收到来自 192.168.2.0 子网的数据，转发到 接口 1 (ens0)
            // (mac1 是 ens0 的 MAC，将作为转发包的源 MAC)
            handle_packet(sd2, sd1, if_index1, mac1, "Router (IF2->IF1)");
        }
    }
    close(sd1);
    close(sd2);
    return 0;
}
```

实验结果：

验收部分1

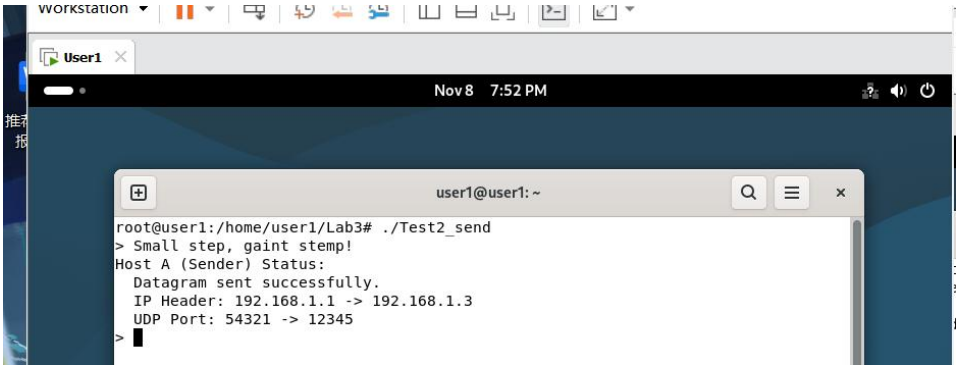

User1
Nov 8 7:47 PM
user1@user1: ~
user1@user1:~$ su
Password:
root@user1:/home/user1# sudo ip route add default via 192.168.1.2 dev ens33
root@user1:/home/user1# cd Lab3/
root@user1:/home/user1/Lab3# gcc -o Test1_send Test1_send.c
root@user1:/home/user1/Lab3# ./Test1_send
UDP Sender (Host A: 192.168.1.1) is running.
Sending to Host B (192.168.1.2:12345). Type 'exit' to quit:
> Hello! Hello! This is Nanami speaking!
[2025-11-08 19:45:45] Message sent: "Hello! Hello! This is Nanami speaking!"
>


User3
Nov 8 7:48 PM
user1@user1: ~
link/ether 00:0c:29:a8:d2:95 brd ff:ff:ff:ff:ff:ff
altname enp2s1
altname enx000c29a8d295
inet 192.168.1.2/24 brd 192.168.1.255 scope global ens33
valid_lft forever preferred_lft forever
inet6 fe80::20c:29ff:fea8:d295/64 scope link proto kernel_ll
valid_lft forever preferred_lft forever
3: ens37: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
link/ether 00:0c:29:a8:d2:9f brd ff:ff:ff:ff:ff:ff
altname enp2s5
altname enx000c29a8d29f
inet 192.168.2.1/24 brd 192.168.2.255 scope global ens37
valid_lft forever preferred_lft forever
inet6 fe80::20c:29ff:fea8:d29f/64 scope link proto kernel_ll
valid_lft forever preferred_lft forever
root@user1:/home/user1/Lab3# gcc -o Test1_forward Test1_forward.c
root@user1:/home/user1/Lab3# ./Test1_forward
UDP Forwarder (Host B: 192.168.1.2) listening on port 12345...
Will forward to Host C (192.168.1.3:54321).
[2025-11-08 19:45:45] Received from 192.168.1.1: "Hello! Hello! This is Nanami s
peaking!". Forwarding to Host C...
Datagram forwarded.

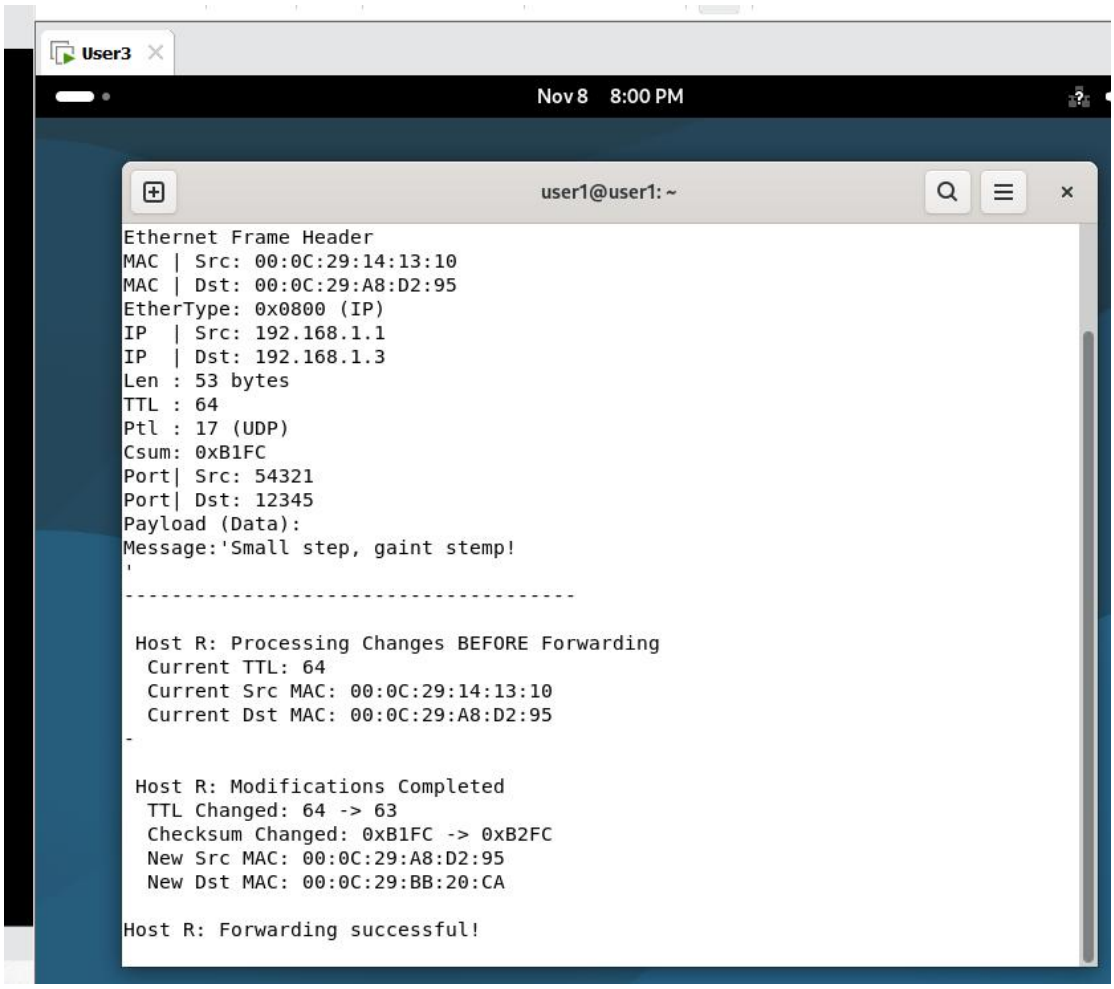


从上图可见，键入的消息从User1发至User3再发至User2，转发成功。

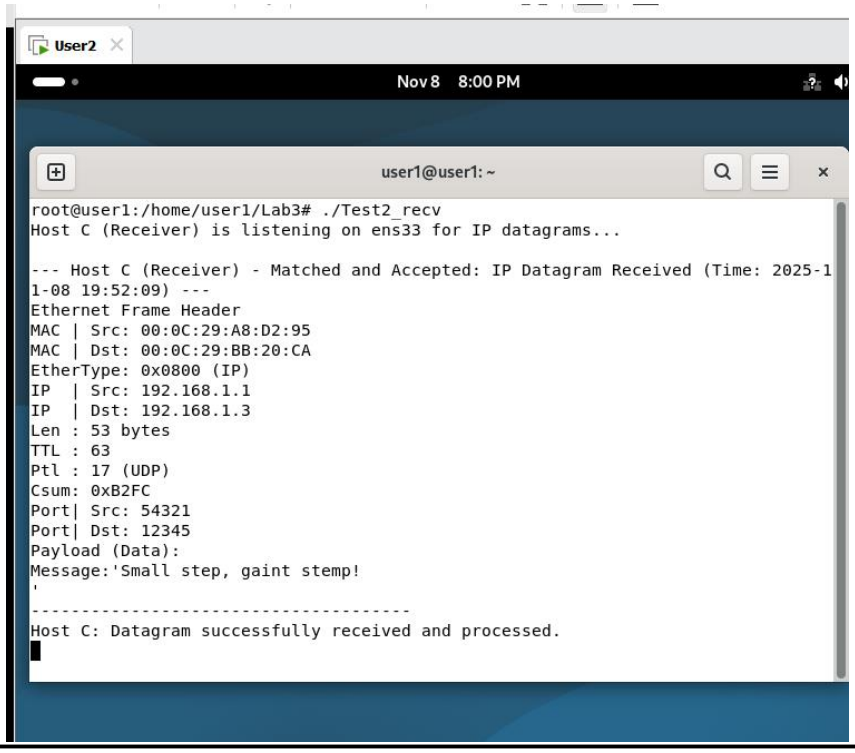
验收内容2



输入信息后，终端打印成功发送，并指出了发送的源IP地址、目的IP地址和对应的端口号。

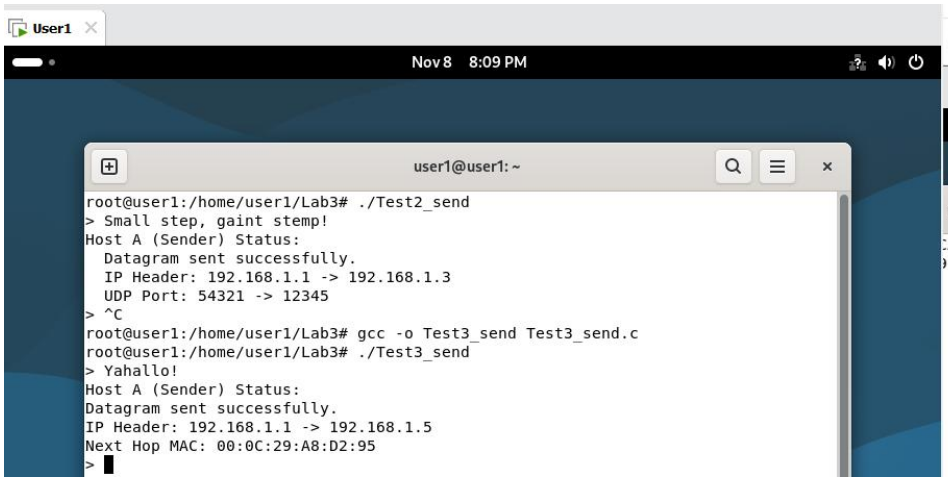


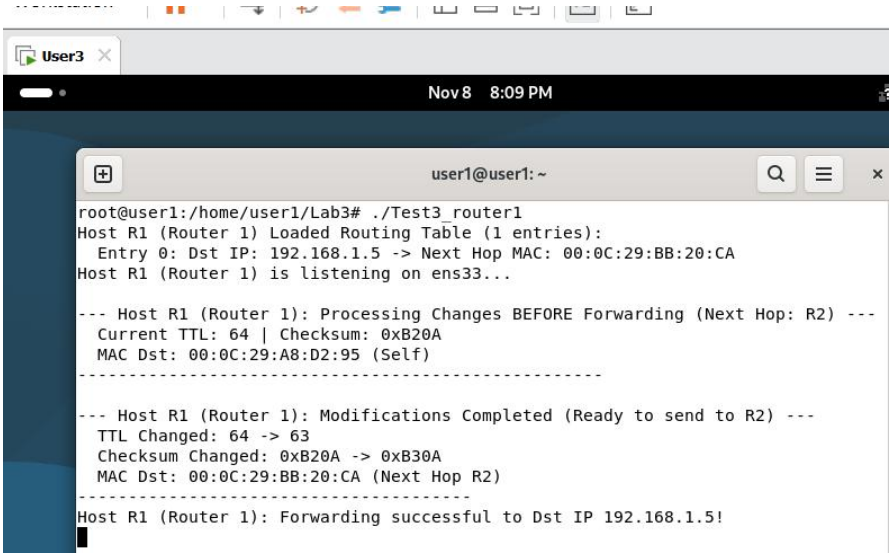
打印了接收到的数据报的完整信息。强调显示User3主机转发前后首部信息的变化，比如TTL减1、MAC地址改变和检验和的变化。

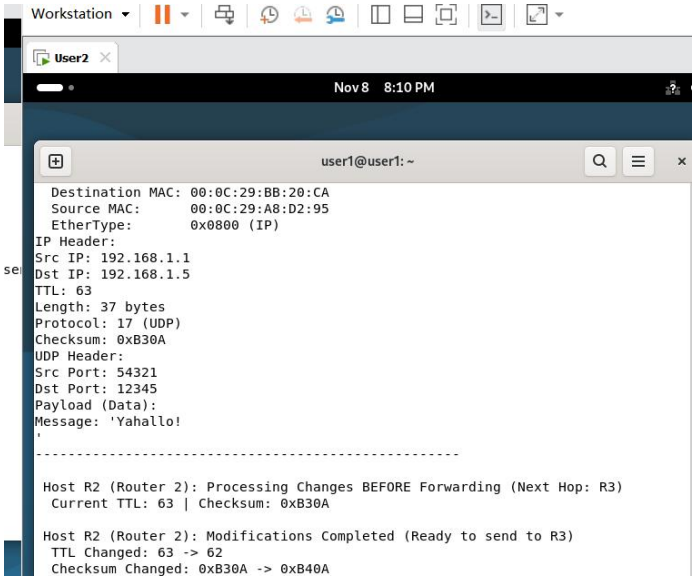


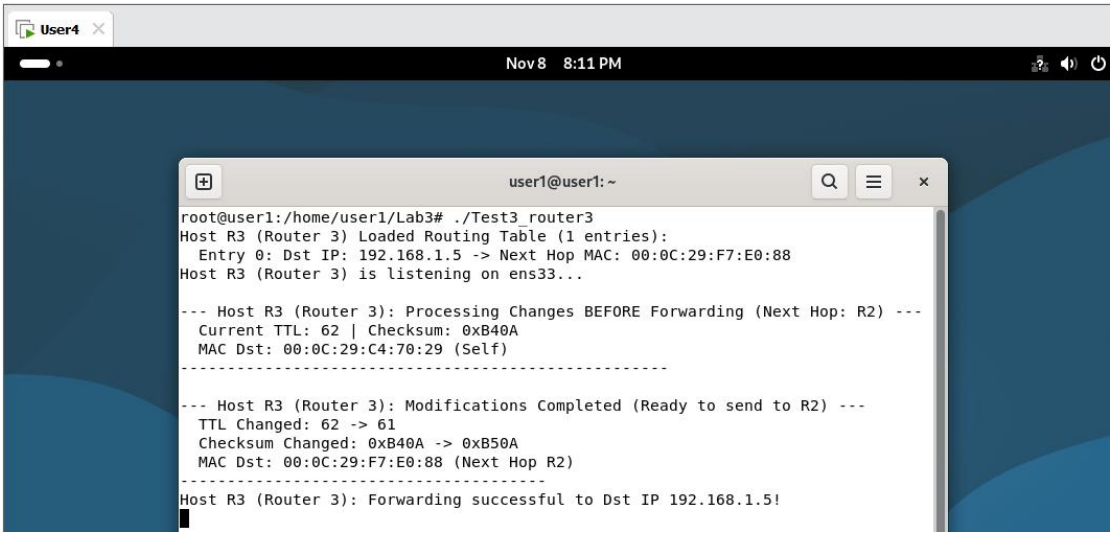
最终在User2主机上打印出了接收到数据报情况。

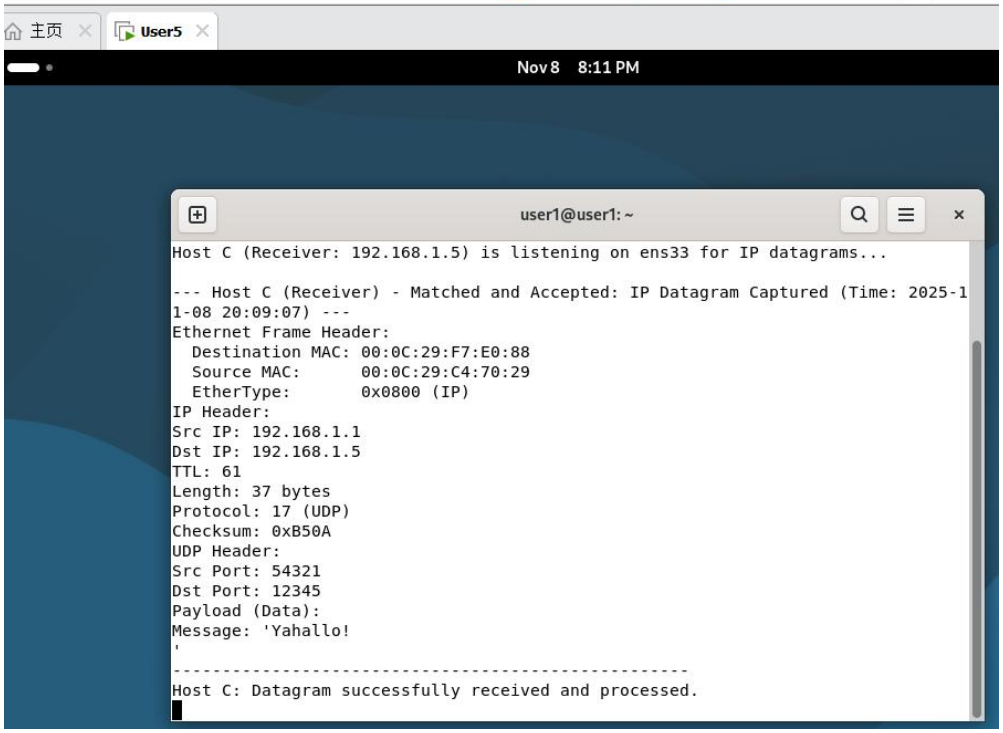
验收内容3





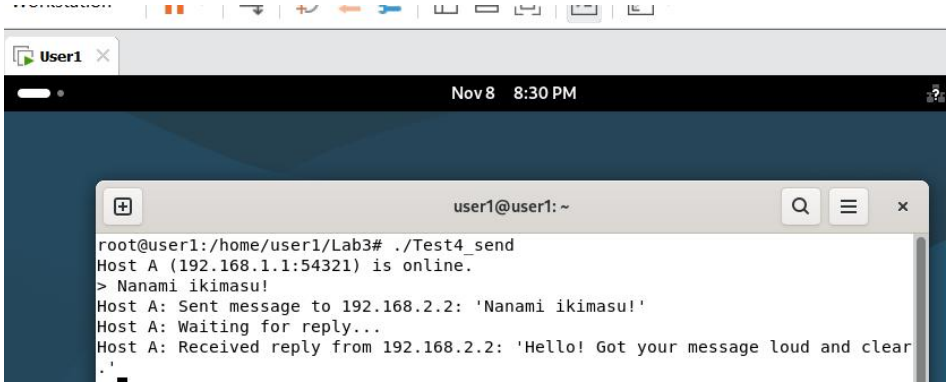


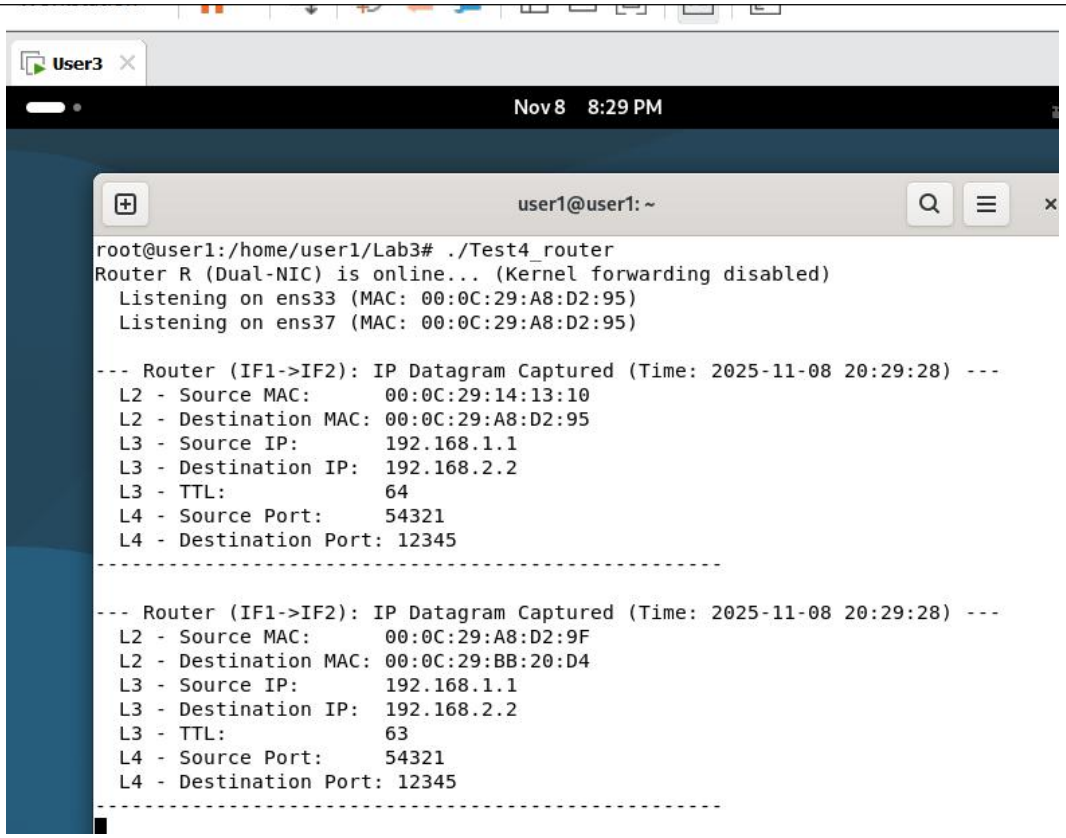


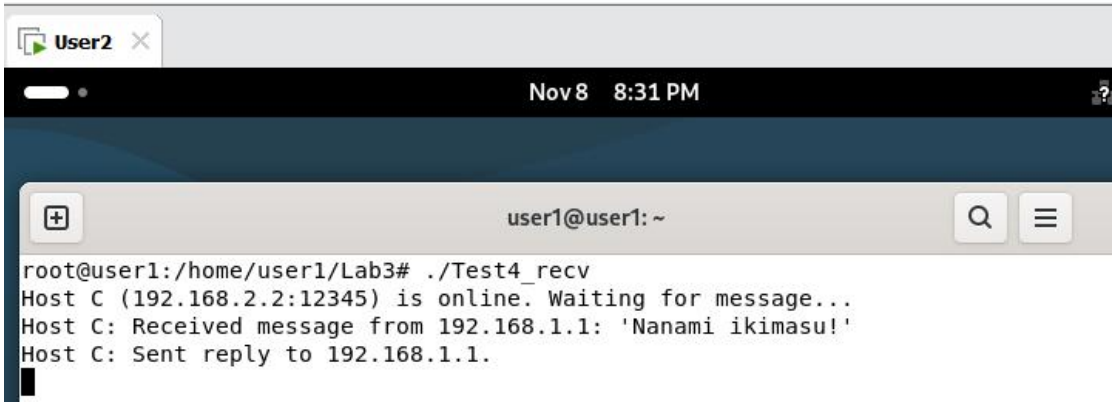


完整展现了数据报从192.168.1.1发送到192.168.1.5的数据报变化的情况，符合设置，验证实验成功。

验收部分4







如图所示，输入的信息成功经路由主机User3到达User2。

问题讨论：

实验开始，发现如果不加以处理，会接收到众多噪声干扰，影响实验观测。选择 AF_PACKET 开启了网卡接口上的监听权限，，局域网中所有来源、所有协议的数据包全部捕获。控制台被来自其他网络设备的 ARP 请求、DHCP 广播所淹没，这严重阻碍了我们对目标数据包的识别与调试。最初的接收错误正是源于这种无序的流量，所以必须手动在程序中重新构建内核本应提供的多层过滤机制，通过精确比对目的 MAC 地址、以太网协议类型、IP 头部和 UDP 目的端口，才得以从洪流中筛选出实验所需的 IP 数据报，确保了接收程序的逻辑焦点和运行效率。

由于网络环境配置时出现疏忽，UFW 防火墙开启导致一部分 UDP 流量被阻隔，无法接收到

数据报，也影响了实验过程。
心得体会：
本次基于原始套接字的 IP 数据报转发模拟实验，成功验证了数据包在逻辑主机 S、路由器 R 和逻辑主机 D 之间的完整传输与转发流程，深入了解了网络协议栈的底层，本次实验是一次对网络模型结构的良好验证，从实验过程中我们切实体会到分层设计接口将程序员的责任从应用层直接下放到了链路层。本次实验要求理解数据报的头部格式，掌握路由器进行 TTL 递减和校验和重算等核心转发逻辑，又再次帮助我们巩固了课上的知识。实验过程中出现的一些问题也启示我们需要处理好外部环境的噪声（过滤）、操作系统与硬件的干扰（防火墙流量阻止）。每解决一个问题，都是对网络协议、C 语言内存模型以及 Linux 内核底层行为的一次体会。